Recovery-Set Contracts for Cryptographic Storage Migration

Supplementary Material and Reproduction Guide

Ishan Teredesai Tushar Rahangdale

8 November 2026

1 Scope and reviewer access

This artifact contains a bounded reference model, a process-interruption harness, and a two-worker concurrent-admission harness. The implementations use SQLite, AES-256-GCM, and an unprotected local JSON key file. That file is a test fixture, not a secure KMS. There is no ML-KEM/hybrid-KEM implementation, QRNG, industrial case study, power-loss experiment, performance benchmark, or cryptographic proof. All data are synthetic. Counts describe selected cases, not failure probabilities.

The private repository is `https://github.com/titcsl/storage-recovery-contracts`. A private URL alone does not grant reviewer access. The complete ZIP includes source, raw observations, traces, manifests, this guide, and current LaTeX/PDFs; it can be transferred through the venue's permitted artifact mechanism without repository access. No public access or reviewer invitation is claimed.

2 Reproduction commands

Use Python 3.10+ on Linux (Windows: WSL/Ubuntu). Unzip the complete artifact and run commands from its `recovery-contracts/` directory. The model needs only the standard library; both storage harnesses require `cryptography`.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
# One command for each experiment:
python3 experiment/model.py --out results
python3 experiment/crash_harness.py --out results
python3 experiment/concurrency_harness.py --out results
# Validate regression tests and consistency of recorded results:
python3 -m unittest discover -s experiment -p 'test_*.py' -v
python3 experiment/verify_results.py --results results
# Or run the entire sequence:
bash run_experiments.sh
```

Commands replace files in `results/`. Preserve the delivered directory and rerun in a separate copy to compare environments. A nonzero exit is a failed run; do not treat old files left in the directory as a new successful run.

Recorded successful environment

Python	3.12.14 (Clang 22.1.3; build dated 25 August 2026)
SQLite / cryptography	3.53.1 / 50.0.2
Platform	Linux 6.18.57, x86-64, glibc 2.39
Concurrent run completed	8 November 2026, 06:49:15 UTC
Scheduling and lock	Two spawned workers; IPC gates; local `fcntl.flock`

The JSON manifests preserve the full runtime strings, source SHA-256 values, configuration bounds, and failure models. No cloud services are used by the code.

3 Finite model and all eighteen configurations

State $S = (a, L, B, P, K)$ contains active generation, committed live references, optional snapshot, volatile pending targets, and available keys. Initially live and snapshot references are zero, the only key is zero, and no work is pending. The invariant concerns live and retained-snapshot references. The retirement guard additionally protects pending targets to preserve prepared work.

Transition	Precondition	Effect
Activate	Next generation is within the bound.	Durably add key; change active generation.
Prepare i	No pending target; source key exists; live generation differs from active.	Set volatile target to active generation.
Commit i	Target key is available.	Publish target and clear pending entry.
Snapshot	No retained snapshot; live keys available.	Copy committed live references.
Release	Snapshot exists; authorized expiry assumed.	Remove that retention obligation, without migrating it.
Crash	Pending work exists.	Discard pending work, retain durable state.
Retire k	Non-active key with no dependency under the selected rule.	Remove key from available inventory.

Breadth-first exploration ends only at a fixed point. Counts include edges to previously visited states. Intent-only substitutes pending targets for live references and ignores snapshots. Live-only checks committed live references. Recovery-set checks live, snapshot, and pending references. These first two policies are constructed negative controls, not reported product vulnerabilities.

Records	Generations	Policy	States	Edges	Missing L	Missing B	Profile gap
1	2	intent-only	16	31	4	3	2
1	2	live-only	12	23	0	1	2
1	2	recovery-set	11	21	0	0	1
1	3	intent-only	76	202	26	20	14
1	3	live-only	52	137	0	7	10
1	3	recovery-set	43	111	0	0	5
2	2	intent-only	48	115	16	13	6
2	2	live-only	32	79	0	3	6
2	2	recovery-set	29	73	0	0	3
2	3	intent-only	500	1,642	230	202	126
2	3	live-only	288	955	0	55	82
2	3	recovery-set	217	713	0	0	39
3	2	intent-only	154	448	52	45	14
3	2	live-only	102	319	0	7	14
3	2	recovery-set	95	305	0	0	7
3	3	intent-only	3,682	14,499	1,814	1,766	926
3	3	live-only	1,996	8,031	0	403	610
3	3	recovery-set	1,495	6,009	0	0	293

Missing L/B counts states with unavailable live/backup keys; the columns can overlap. Profile gap means live references are target-labeled but a retained snapshot remains legacy. The guarded configurations total 1,890 states and 7,232 edges, counted separately, with no missing-key state. The largest has 293 profile-gap states. None of these quantities is a deployment frequency.

Release is an authorized retention action, not snapshot migration or secure erasure. A guard may preserve reference availability while blocking retirement indefinitely. Labels do not establish actual post-quantum protection.

4 Process interruption and retained snapshots

Each record receives a fresh 32-byte DEK. Payload encryption and DEK wrapping use AES-256-GCM with separate random 12-byte nonces. Payload associated data binds record identity and content version; wrapping associated data binds identity and epoch. The key file uses file synchronization and atomic replacement; SQLite uses rollback journaling with `synchronous=FULL`. These are separate durability domains. The test stops a process, not the kernel or power supply.

The controller provisions generation 1, prepares replacement envelopes, attempts retirement, applies a SQL transaction, commits, and attempts retirement again. A pre-rotation snapshot remains at generation 0. The parent restores both copies using the available key file and checks plaintext bytes against generated input.

Cut/completion point	Intent-only L/B	Live-only L/B	Recovery-set L/B
After durable provisioning	0/0	0/0	0/0
After envelope preparation	0/0	0/0	0/0
After first retirement attempt	6/6	0/0	0/0
After SQL updates, before commit	6/6	0/0	0/0
After database commit	0/6	0/0	0/0
After final retirement attempt	0/6	0/6	0/0
Normal completion	0/6	0/6	0/0

Each cell reports failure counts across six datasets: one/eight records at 512 B, 16 KiB, or 256 KiB each. Every policy/cut/dataset runs once. There are 126 cases, with 108 child terminations at exit code 93 and 18 normal exits. Columns may overlap; these are deterministic coverage outcomes.

Policy	Cases	Live failures	Backup failures	Both restored
Intent-only	42	12	30	12
Live-only	42	0	12	30
Recovery-set	42	0	0	42

An unavailable wrapping key counts as a failed restore. An authentication error, wrong row count, unexpected child exit, or plaintext mismatch aborts the run. The harness records observed restoration counts before checking its expected control behavior; it does not substitute expected counts for decryption results. The separate concurrent harness has no expected-result function.

Minimal model witnesses

For one record, activate 1 and prepare a replacement. Intent-only retirement can then remove 0 while the committed live reference is still 0. A crash clears its volatile replacement. Alternatively, commit before retiring 0 under live-only: the live reference is now 1, but the snapshot still requires 0. The guarded rule rejects that deletion while the snapshot remains in the recovery set.

Different evidence from the two mechanisms

The model interleaves individual record transitions. The interruption harness rewraps a complete dataset in one SQL transaction and interrupts an actual process. The concurrent experiment on the next page adds an admission operation for an earlier epoch choice; it is not a refinement proof of the closed model.

5 Concurrent admission protocol and schedules

The coordinator starts two real worker processes, using Python's `spawn` method. One retires generation 0 and one attempts to prepare and publish references to generation 0. Initially the live database and snapshot reference 1, active generation is 1, both keys exist, and no pending pin exists. Generation 0 is therefore eligible for retirement at the initial scan.

The writer represents an admitted request carrying an earlier target choice, as might occur with delayed work or an import. This is an explicit extension of the finite model: its ordinary prepare operation always selects the active generation. A current-generation-only implementation can reject old choices; this experiment instead admits an inactive choice only while its key exists. It tests reference availability, not whether permitting that choice is a good profile-migration policy. Already registered pending work is included in both implementations' scans.

Actor	Shared-lock implementation: exact protected interval
Retirer	Acquire one exclusive lock; inspect committed live references, retained snapshot, registered pending targets, available keys, and active generation; decide eligibility; remove key if permitted; release lock.
Writer	Acquire the same lock; validate target-key availability; register a pending pin in SQLite; prepare encrypted replacement envelopes; atomically publish live references and remove the pin in one SQL transaction; release lock.
Coordinator	Holds no lifecycle lock; requests actions via IPC and records responses, contention, and inventories. Reopens both copies after the workers finish and checks actual restoration.

Both workers open the same stable lock-file inode. `flock(LOCK_EX | LOCK_NB)` reports contention without timing sleeps. A blocked action is retried only after the holder has completed. The unlocked implementation uses the same dependency scan and key-availability check but omits the shared lock. It can act on a stale scan. Locking only the deletion would not protect the whole check/delete interval.

Activation and snapshot replacement/release are disabled during these trials. A deployment enabling them must synchronize their dependency changes too. The pending registry here is a durable SQLite table; the finite model's pending work is volatile. No worker failure is injected in the concurrent trials, so reclaiming abandoned durable pins is not evaluated. The lock is advisory and local, requiring all participants to cooperate.

Schedule	Requested order (C: check; P: prepare/register; W: commit; D: delete)
`commit_in_gap`	C, P, W, D
`prepare_in_gap`	C, P, D, W
`writer_first`	P, C, W, D
`delete_first`	C, D, P

For a blocked operation, the requested unsafe order is prevented: the holder finishes first and the blocked operation is retried. Four schedules times two protocols times six datasets produce 48 cases. Each runs once. This is controlled interleaving coverage, not exhaustive thread scheduling or a statistical sample.

6 Observed concurrent results and traces

Schedule	Unlocked L/B	Locked L/B	Locked rejected	Locked committed
Commit in gap	6/0	0/0	6	0
Prepare in gap	6/0	0/0	6	0
Writer first	0/0	0/0	0	6
Delete first	0/0	0/0	6	0

L/B counts failed live/backup restores among six datasets per cell. All 24 protected cases restore both copies. In the unlocked implementation, the two gap schedules fail live restoration in all twelve cases. Both snapshots always restore because they keep generation 1. When the writer's registered pin exists before the scan, even the unlocked guard sees it and retains the key.

The following is the observed `prepare_in_gap` trace for one 512-byte record. Vectors are abbreviated by their single generation; K is the available key set and $P = -$ means no pending entry. These tables summarize the actual `concurrency_traces.json` events.

Step	Unlocked observation	Live	Backup	Pending	Keys
0	Initial	1	1	–	0,1
1	Retirer checks: eligible	1	1	–	0,1
2	Writer registers and prepares	1	1	0	0,1
3	Retirer deletes from stale decision	1	1	0	1
4	Writer commits and clears pin	0	1	–	1
5	Restore: live fails; backup succeeds	0	1	–	1

Step	Shared-lock observation	Live	Backup	Pending	Keys
0	Initial	1	1	–	0,1
1	Retirer locks and checks: eligible	1	1	–	0,1
2	Writer's lock attempt is blocked	1	1	–	0,1
3	Retirer deletes and unlocks	1	1	–	1
4	Writer retries: target key absent	1	1	–	1
5	Restore: both copies succeed	1	1	–	1

The raw traces additionally include stop acknowledgments, every inspected inventory, lock ownership, cached eligibility, pre-deletion inventory, and exact per-case restore counts. Sequence numbers are coordinator event order, not latency measurements. Worker command timeouts and nonzero exits abort a run.

Interpretation and limits

When retirement wins the shared lock, the delayed writer is safely rejected. When the writer wins, it commits while holding the lock; retirement later sees the committed dependency and declines deletion. This preserves availability for these schedules. It does not guarantee eventual retirement or target-profile completion. Uncooperative writers, distributed key stores, changing snapshot inventories, worker crashes while holding pins, and all other schedules remain outside the concurrent evaluation.

7 File-to-claim map and validation

Paths below are relative to the unzipped artifact root.

File	Paper table/claim supported
`results/model_summary.csv`	Table II, including largest-bound state counts.
`results/model_results.json`	All eighteen configurations and guarded totals.
`results/model_traces.json`	Shortest missing-key and profile-gap witnesses.
`results/crash_trials.csv`	Table III: 126 individual interruption/completion cases.
`results/crash_summary.json`	Table III's policy aggregates.
`results/concurrency_trials.csv`	Table IV: 48 observed concurrent cases.
`results/concurrency_summary.json`	Table IV's protocol/schedule aggregates.
`results/concurrency_traces.json`	Ordered observations and restores for every concurrent case.
`results/*_manifest.json`	Runtime versions, source hashes, configurations, and scope.
`results/execution_log.txt`	Successful regression and experiment console output.
`experiment/verify_results.py`	Checks recorded source hashes, totals, and agreement among result files.
`VALIDATION.md`	Claim corrections, reference audit, and review-response mapping.
`paper/paper.tex`, `paper.pdf`	Current manuscript with repository URL.
`paper/supplement.tex`, `supplement.pdf`	This guide and its source (both under `paper/`).

Tables I and V are source comparison and illustrative acceptance obligations, not measured outcomes. The figure explains the interruption sequence; it is not a performance plot. All tables are self-contained in the main manuscript.

Expected reproduction totals and failure conditions

Nine tests pass: the six original model checks plus late-dependency races, registered-pending guard coverage, and the four protected schedules. Exploration produces eighteen configurations. The interruption harness produces 126 cases (108 terminations), with policy live/backup failures 12/30, 0/12, and 0/0. The concurrent harness produces 48 cases, with unlocked live failures 12/24, protected live failures 0/24, and no backup failure. These totals describe executed reference runs and are checked only after observations are generated.

A timeout, failed worker, unexpected exit, authentication error, wrong row count, plaintext mismatch, dangling pin, or inventory/restore disagreement is an error. A protected missing-key state or disagreement between raw files and summaries also fails validation. Negative-control failures are intentional evidence and must not be silently converted into successful restores.

Integrity, build, and provenance

```
sha256sum --check SHA256SUMS
cd paper
latexmk -pdf -interaction=nonstopmode -halt-on-error paper.tex
latexmk -pdf -interaction=nonstopmode -halt-on-error supplement.tex
```

The unchanged IEEEtran class is included with its license notices; a TeX Live installation with TikZ and Latin Modern builds the documents. Rerunning can change manifests (including the completion timestamp), and rebuilding PDFs can change embedded dates. Preserve the delivered package before such operations. OpenAI ChatGPT assisted with the code, model, diagram, text, and validation. Results were obtained by executing the supplied programs. Author review and adoption remain necessary; this artifact does not certify a production system.